

Vector Databases Benchmarking for Multimodal Semantic Search

Intermediate Report, CSCI-Y 790 Graduate Independent Study

Shivali Mate

Luddy School of Informatics, Computing & Engineering
Indiana University Bloomington

Advisor: Prof. Feng Chen

Department of Computer Science
Indiana University Bloomington

Abstract—Vector databases have become a fundamental component of modern information retrieval systems, particularly in semantic search and retrieval augmented generation pipelines. This work presents a systematic benchmarking study of vector databases for multimodal retrieval tasks. A modular pipeline is developed to evaluate ChromaDB, Qdrant, and Milvus across multiple indexing strategies, including HNSW, IVF FLAT, and DiskANN. The system processes a corpus of 1,000 Wikipedia articles and extends to a multimodal setting using image text pairs from the Wikipedia Image Text (WIT) dataset. Text and image data are embedded into a shared vector space using CLIP, enabling cross modal retrieval. Performance is evaluated using latency and throughput metrics over natural language queries. The proposed framework provides a reproducible foundation for evaluating vector database systems under realistic multimodal workloads.

Index Terms—vector databases, semantic search, HNSW, DiskANN, IVF FLAT, ChromaDB, Qdrant, Milvus, CLIP, multimodal retrieval

I. INTRODUCTION

THE increasing adoption of large language models and retrieval augmented generation [1] systems has led to a growing reliance on vector databases for efficient semantic search. Unlike traditional keyword based retrieval, vector databases operate on dense embeddings and enable approximate nearest neighbor search [2], allowing systems to retrieve results based on semantic similarity rather than exact token matching.

Despite the rapid development of vector database systems such as ChromaDB, Qdrant, and Milvus, there is limited systematic comparison [3] of their performance under controlled and reproducible settings. Existing evaluations often focus on a single database or indexing strategy, making it difficult to understand how design choices impact performance across systems.

This work addresses this gap by developing a modular benchmarking pipeline that evaluates multiple vector databases under identical experimental conditions. The study compares different indexing strategies, including HNSW, IVF FLAT, and DiskANN, and analyzes their impact on query latency and throughput.

In addition to text based semantic search, the system is extended to support multimodal retrieval using CLIP embeddings. By integrating image text pairs from the Wikipedia

Image Text dataset, the pipeline enables cross modal retrieval, allowing text queries to retrieve images and vice versa.

The key contribution of this work is a unified framework for benchmarking vector databases across multimodal settings. Preliminary results show that indexing strategies play a critical role in performance, with graph based methods like HNSW achieving lower latency, while disk based approaches like DiskANN provide better scalability for larger datasets.

II. SYSTEM ARCHITECTURE

A. Pipeline Overview

The benchmarking pipeline follows a modular, database agnostic design. Fig. 1 illustrates the end-to-end flow for multimodal retrieval. The architecture enables consistent execution across multiple vector databases by abstracting database specific operations through interchangeable connectors. This ensures a fair comparison of performance across systems.

B. Data Ingestion and Chunking

The corpus consists of 1,000 English Wikipedia articles sampled from the Hugging Face wikipedia dataset. Each article is stored as a separate text file identified by its corresponding page ID to ensure reproducibility between runs.

Text documents are split into overlapping chunks (500 characters with 50 character overlap) to preserve contextual continuity while maintaining fine grained retrieval units.

For multimodal experiments, 1,000 pairs of image text are incorporated from the Wikipedia Image Text (WIT) dataset [8]. Each image is associated with metadata including captions, keywords, image identifiers, and corresponding article identifiers. This metadata is retained alongside embeddings to support retrieval and evaluation.

C. Embedding Generation

Embeddings are generated locally using transformer-based models. For text-only experiments, the sentence-transformers library `all-MiniLM-L6-v2` [6] produces 384-dimensional vectors.

For multimodal retrieval, CLIP (`clip-ViT-B-32`) [5], is used to encode text and images into a shared 512-dimensional vector space. This enables direct comparison between modalities and supports cross-modal retrieval tasks such as text-to-image and image-to-text search.

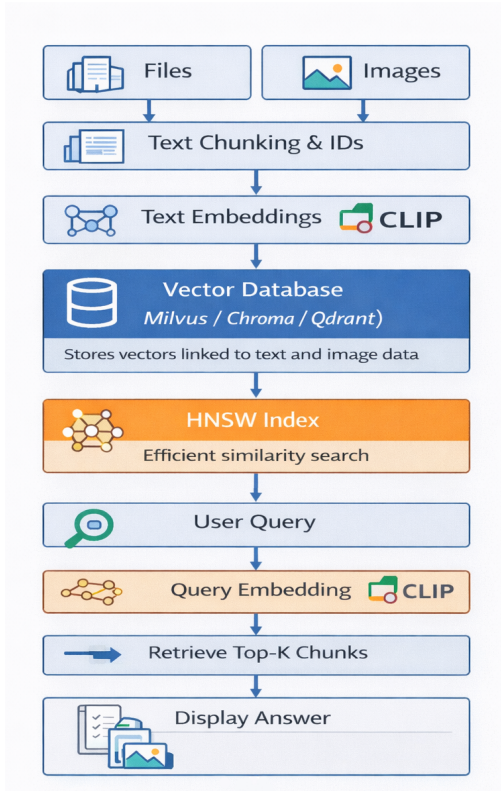


Fig. 1. Multimodal semantic search pipeline.

D. Vector Storage and Metadata Mapping

Each embedding is stored along with a metadata payload that contains relevant identifiers and descriptive information. For text data, this includes the document ID and chunk offset. For image data, metadata includes captions, keywords, image ID, and associated article ID.

This pointer based design allows retrieved results to be mapped back to their original content without storing full documents within the vector index. The inclusion of metadata also supports interpretation of results and enables evaluation of retrieval quality.

E. Indexing

Multiple approximate nearest neighbor indexing strategies are evaluated:

- **HNSW** (Hierarchical Navigable Small World) [4], a graph-based index enabling fast in-memory similarity search, used in ChromaDB, Qdrant, and Milvus
- **IVF_FLAT**, an inverted-file flat index that partitions cluster vectors into regions and performs search within candidate partitions, used in Milvus
- **DiskANN** [9], a disk-based ANN index optimized for large-scale datasets, enabling efficient search with reduced memory usage

These indexing strategies allow comparison between in-memory and disk-based search approaches under identical workloads.

F. Query Evaluation

A set of 1,000 natural language queries is used for evaluation. Each query is embedded using the same model as used during ingestion, and top-K results are retrieved from the vector database.

Performance metrics including total execution time, average latency, and throughput are recorded across all queries. Retrieved results include associated metadata, enabling interpretation of both text and image outputs.

III. ENVIRONMENT AND SETUP

A. Hardware

Development was performed on a local Windows 11 workstation. Large scale experiments requiring Milvus Standalone and Docker were run on **IU Research Desktop (RED)**, a Linux based remote computing environment provided by Indiana University.

B. Software Stack

- **Language:** Python 3.12 (virtual environment)
- **Embeddings:** sentence-transformers 2.x, torch (CPU), transformers, OpenAI CLIP
- **Vector DBs:** chromadb, qdrant-client, pymilvus
- **Data:** datasets (Hugging Face), numpy, pandas

C. ChromaDB Setup

ChromaDB is configured as the initial vector database in the benchmarking pipeline. Collections are created with embeddings of dimensionality 384 for text data, using cosine similarity as the distance metric. HNSW indexing is used by default for efficient in-memory similarity search.

Embeddings and associated metadata are stored using persistent storage, allowing reuse across multiple runs without reindexing. Data is inserted in batches to support efficient ingestion.

D. Qdrant Setup

Qdrant is integrated using its Python client and configured with a collection schema specifying vector dimensionality and cosine distance. HNSW indexing is used for approximate nearest neighbor search.

Data is inserted using structured points, where each vector is associated with a payload containing metadata such as document identifiers, captions, and keywords. The system uses a unified query interface to retrieve top-K results across queries.

E. Milvus Setup

Milvus is configured to evaluate multiple indexing strategies within the same framework [10]. Collections are created with appropriate vector schemas, and embeddings are inserted along with metadata.

Three indexing strategies are evaluated namely IVF FLAT, which partitions the vector space into clusters and performs search within candidate partitions. HNSW, which provides fast

in-memory graph-based similarity search. DiskANN, which enables disk-based approximate nearest neighbor search optimized for large scale datasets.

Collections are indexed and loaded prior to query execution to ensure efficient retrieval performance.

F. Dataset and Query Setup

The dataset consists of 1,000 Wikipedia text documents and 1,000 image text pairs from the WIT dataset. Text data is processed into overlapping chunks, while image data is stored along with associated metadata including captions, keywords, image identifiers, and article identifiers.

A set of 1,000 natural language queries is used for benchmarking. Each query is embedded and evaluated using top-K retrieval across all database configurations.

IV. PRELIMINARY BENCHMARKING RESULTS

All benchmarks are evaluated over a fixed set of 1,000 natural language queries. Latency refers to the average time taken to complete a single query, measured in seconds. Throughput refers to the number of queries processed per second. Table I summarizes the results across database and index configurations.

TABLE I
QUERY PERFORMANCE ACROSS VECTOR DATABASES

Database (Index)	Total Time (s)	Avg Lat. (s)	Throughput (q/s)
Milvus (HNSW)	8.97	0.0048	111.51
Milvus (DiskANN)	11.04	0.0073	90.58
Milvus (IVF)	10.32	0.0068	96.89
Qdrant (HNSW)	8.54	0.0046	117.09
Chroma (HNSW)	9.12	0.0050	109.65

V. FUTURE WORK

Retrieval quality evaluation (Recall@K). Future work will incorporate retrieval quality metrics in addition to latency and throughput. Recall@K will be used to measure whether relevant results are retrieved within the top-K outputs.

For text retrieval, semantic similarity between queries and retrieved documents will be evaluated using transformer based models. For image retrieval, metadata such as captions and keywords will be compared against ground truth answers. To improve robustness beyond keyword matching, BERT based similarity measures will be used to capture semantic alignment between retrieved metadata [1] and expected outputs.

Database scaling experiments. Current experiments use 1,000 documents. Future runs will scale the corpus to 10,000 and 100,000 documents to evaluate how latency and recall degrade (or hold) as index size grows. Scaling behavior is particularly important for DiskANN, which is optimized for billion scale datasets.

Accessing datasets from a remote database. Rather than loading Wikipedia files from local disk, a planned enhancement will stream documents directly from a hosted database, decoupling the benchmarking pipeline from local storage constraints and enabling experiments on larger corpora without full local download.

VI. CONCLUSION

This midterm report documents the design and partial implementation of a modular vector database benchmarking framework. Three databases ChromaDB, Qdrant, and Milvus, have been integrated and evaluated on a document Wikipedia corpus with natural language queries. Preliminary latency and throughput results show meaningful differences across systems even before recall evaluation is complete. The system is well positioned for the planned extensions: multimodal CLIP retrieval over the WIT dataset, dataset scaling experiments. The modular architecture ensures that each of these extensions can be added without modifying the core benchmarking logic.

REFERENCES

- [1] P. Lewis *et al.*, “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [2] J. Johnson, M. Douze, and H. Jégou, “Billion-scale Similarity Search with GPUs,” *IEEE Trans. Big Data*, vol. 7, no. 3, pp. 535–547, 2021.
- [3] M. Aumüller, E. Bernhardsson, and A. Faithfull, “ANN-Benchmarks: A Benchmarking Tool for Approximate Nearest Neighbor Algorithms,” *Information Systems*, vol. 87, 2020.
- [4] Y. A. Malkov and D. A. Yashunin, “Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 42, no. 4, pp. 824–836, 2020.
- [5] A. Radford *et al.*, “Learning Transferable Visual Models From Natural Language Supervision,” in *Proc. ICML*, 2021, pp. 8748–8763.
- [6] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” in *Proc. EMNLP*, 2019, pp. 3982–3992.
- [7] T. Kwiatkowski *et al.*, “Natural Questions: A Benchmark for Question Answering Research,” *Trans. Assoc. Comput. Linguistics*, vol. 7, pp. 452–466, 2019.
- [8] K. Srinivasan *et al.*, “WIT: Wikipedia-based Image Text Dataset for Multimodal Multilingual Machine Learning,” in *Proc. ACM SIGIR*, 2021, pp. 2443–2449.
- [9] S. Jayaram Subramanya *et al.*, “DiskANN: Fast Accurate Billion-point Nearest Neighbor Search on a Single Node,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [10] J. Wang *et al.*, “Milvus: A Purpose-Built Vector Data Management System,” in *Proc. ACM SIGMOD*, 2021, pp. 2614–2627.